

CPSC 250

Classes and Objects

Edward Brash

1 Object-Oriented Programming

Object-oriented programming is a style of programming based on the idea that a program can be organized around objects.

An object combines:

- data,
- functions that operate on that data,
- and rules for how outside code should interact with that data.

This gives us a powerful way to manage complexity.

2 Concept 1: Encapsulation

One of the central ideas of object-oriented programming is **encapsulation**.

Encapsulation means hiding internal variables from the user and requiring the user to interact with an object through well-defined methods.

2.1 Why Encapsulation Is Good

Encapsulation helps with:

- maintenance,
- debugging,
- team software development,
- changing internal implementation details without breaking user code.

The idea is that the user of a class should not need to know exactly how the class works internally. They should only need to know how to use its public methods.

3 Good and Bad Design

Good design hides unnecessary details.

Bad design exposes too much internal structure and forces users to depend on details that should not matter.

A good object acts like a carefully designed interface.

The user should interact with the object through functions, not by manually changing internal data.

4 Objects and Classes

4.1 Objects

An object is a grouping of variables and functions that act on those variables.

4.2 Classes

A class is a template used to create objects.

A class defines:

- what variables an object will contain,
- what functions operate on those variables,
- and how the object should behave.

In this sense, a class is the DNA of object-oriented programming.

5 Example: A Restaurant Object

Suppose we want to build a restaurant rating system, similar in spirit to Yelp.

For each restaurant, we might want to store:

Information	Example	Type
Name	Moe's	string
Rating	7	integer
Price	***	string
Cuisine type	Mexican	string

In addition, we might want functions that act on those variables:

- set the name,
- get the name,
- set the rating,
- get the rating,
- set the price,
- get the price,
- set the cuisine,
- get the cuisine,
- print a report.

6 Defining a Restaurant Class

A first version of the class might look like this:

```
class Restaurant:

    def __init__(self):
        self.name = "none"
        self.rating = -1
        self.price = "none"
        self.cuisine = "none"

    def set_name(self, user_name):
        self.name = user_name

    def get_name(self):
        return self.name

    def set_rating(self, user_rating):
        self.rating = user_rating

    def get_rating(self):
        return self.rating

    def set_price(self, user_price):
        self.price = user_price

    def get_price(self):
        return self.price

    def set_cuisine(self, user_cuisine):
        self.cuisine = user_cuisine

    def get_cuisine(self):
        return self.cuisine

    def print_info(self):
        print("Restaurant Information")
        print(f"Name: {self.name}")
        print(f"Rating: {self.rating}")
        print(f"Price: {self.price}")
        print(f"Cuisine Type: {self.cuisine}")
```

7 Using the Class

In the main program, we can create an object:

```
moes = Restaurant()
```

Then we can interact with the object using methods:

```
moes.set_name("Moe's")
moes.set_rating(7)
moes.set_price("***")
moes.set_cuisine("Mexican")
```

```
moes.print_info()
```

8 The Constructor

The method:

```
def __init__(self):
```

is called the constructor.

It is called automatically whenever a new object of the class is created.

9 The Meaning of self

Every method in a class usually has **self** as the first parameter.

The variable **self** refers to the particular object that is calling the method.

For example:

```
moes.set_name("Moe's")
```

means that inside **set_name**, **self** refers to **moes**.

Thus:

```
self.name = user_name
```

means:

Set the name belonging to this particular object.

10 Public Access vs Good Practice

In Python, instance variables are technically public unless special naming conventions are used.

That means it is possible to write:

```
print(moes.name)
```

However, this is bad practice in introductory object-oriented design.

Instead, prefer:

```
print(moes.get_name())
```

and:

```
moes.set_name("Moe's")
```

This preserves encapsulation.

11 Instance Attributes

Variables such as:

- **self.name**,

- `self.rating`,
- `self.price`,
- `self.cuisine`

are called **instance attributes**.

They belong to each individual object.

12 Class Attributes

A class attribute belongs to the class as a whole rather than to a single object.

For example:

```
class MarathonRunner:

    race_distance = 42.195

    def __init__(self):
        self.name = "none"
        self.time = 0.0
```

Here, `race_distance` applies to all marathon runners.

```
runner1 = MarathonRunner()
runner2 = MarathonRunner()

print(runner1.race_distance)
print(runner2.race_distance)
print(MarathonRunner.race_distance)
```

All three print the same class-level value.

13 Key Takeaways

- A class is a template for creating objects.
- An object contains data and methods.
- Encapsulation hides internal details.
- Constructors initialize objects.
- `self` refers to the current object.
- Instance attributes belong to individual objects.
- Class attributes belong to the class as a whole.